Pytorch Autodifferentiation

Derivative of Loss gives you gradient
- ↳ If i change a weight tiny amount how will the loss change

$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$ → Derivative of Loss / weights

```
w = torch.randn(5, 3, requires_grad=True)
```

Creates Random Weights

Tells pytorch to keep track of everything and then calculate derivatives for it

```
Python

loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

Model predicted z
Correct answer y
Compares it and finds the loss

Once get loss question becomes what weights cause loss

Computational graph
- Trace of where the loss came from

loss.backwards = Back propogation
- ↳ work backwards from graph to calculate gradients
  - ↳ Recreates weights for weights / Bias

$L(w) = w^2$

$L'(w) = 2w \quad w = 4$

$L'(w) = 8$ → Inc weight, inc loss

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

→ Erases Gradients
→ Process Image / Forward Pass
→ Calculates gradients
→ Updates weights

$XW + b$ = Produce one Logit

X = Input [Matrix]

W = Matrix of weights for neurons

b = bias

Leaf Node: An original tensor that was created / Pytorch tracked it

If output in Vector use Jacobian matrix